

Table of contents

Serie 2	1
Methods	1
Method 1: Syntax Analysis	1
Method 2: Operational Semantics	1
Method 3: Formal Specification with First-Order Logic (FOL)	1
Method 4: Data Structure Modeling (Arrays)	2
Reminder	2
Illustrative Examples	2
Example 1: Syntax and Contextual Analysis	2
Example 2: Operational Semantics with Valuation and Memory	3
Example 3: FOL Specification and Array Modeling	3
Synthesis	4

Serie 2

Methods

Method 1: Syntax Analysis

- **Context-free grammar (CFG) verification:** identification of basic language structure errors (parentheses, punctuation, keywords, token order).
- **Contextual verification:** identification of errors that depend on context (undeclared variables, incompatible types, scope rules).

Method 2: Operational Semantics

- **Evaluation by transition rules:** use of valuation functions (**Val**) and memory (**Mem**) to describe step-by-step program execution.
- **Expression evaluation:** calculation of an expression's value in a given memory state.
- **Memory update:** modification of the memory state during assignment or other actions.

Method 3: Formal Specification with First-Order Logic (FOL)

- **Preconditions:** conditions on inputs that must be true before execution.
- **Postconditions:** conditions on outputs/memory that must be true after execution.
- **Formalization of properties:** use of quantifiers, logical and arithmetic operators to express invariants, goals.

Method 4: Data Structure Modeling (Arrays)

- **Array representation:** use of `a.length` and `a[i]` to manipulate arrays.
- **Array traversal:** iteration over indices with `while` or `for` loops.
- **Transformation specification:** informal description then formalization in FOL.

Reminder

- **Syntax/semantics distinction:** syntax concerns form, semantics concerns meaning.
- **Operational semantics:**
 - `Val(e, )` gives the value of expression `e` in memory state
 - `Mem(P, )` gives the memory state after executing program `P` from .
- **Logic laws:**
 - Implication elimination: $(P \rightarrow Q) \wedge P \rightarrow Q$
 - Distributivity: $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$, etc.
 - De Morgan: $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$, $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
- **Preconditions/postconditions:** partial correctness (if the precondition is true and the program terminates, then the postcondition is true); termination must be treated separately.
- **Loop invariants:** property that remains true before and after each iteration, used to prove loop correctness.

Illustrative Examples

Example 1: Syntax and Contextual Analysis

Context: Exercise 1 – error detection in a `remainder` program.

Application:

- The given program is:

```
remainder(a,b) = aux := b; while (res >= aux) res := res - aux; done res;
```
- **Context-free grammar error:** the semicolon after `res := res - aux` is incorrect according to the described syntax (probably a misplaced instruction separator). Moreover, the `while` loop should have a delimited instruction block (e.g., with `do ... done`), but here a `do` is likely missing after the condition.
- **Contextual error:** the variable `res` is used but was never declared or initialized. Similarly, `aux` is declared but `res` is not. This violates a contextual rule (every variable must be declared before use).

Example 2: Operational Semantics with Valuation and Memory

Context: Exercise 2 – evaluation of $\text{abs}(a,b)$ in state $\sigma = \{a \mapsto 3, b \mapsto 9\}$.

Application:

- We assume that $\text{abs}(a,b)$ is defined as:

```
abs(a,b) = res := 0; if (a < b) then res := b - a else res := a - b endif; res
```

- Step by step:

1. $\text{Val}(\text{abs}(3,9), \sigma) = \text{Val}(\text{abs program}; \text{res}, \sigma)$
2. We evaluate the program: $\text{Mem}(\text{abs program}, \sigma)$
3. $\text{Mem}(\text{res} := 0, \sigma) = [\text{res} \mapsto \text{Val}(0, \sigma)] = \sigma' = \{a \mapsto 3, b \mapsto 9, \text{res} \mapsto 0\}$
4. We evaluate the if: $\text{Val}(a < b, \sigma') = (\text{Val}(a, \sigma') < \text{Val}(b, \sigma')) = (3 < 9) = \text{True}$
5. So we execute $\text{res} := b - a$: $\text{Mem}(\text{res} := b - a, \sigma') = \sigma''[\text{res} \mapsto \text{Val}(b - a, \sigma')] = \sigma'' = \{a \mapsto 3, b \mapsto 9, \text{res} \mapsto 6\}$
6. Then, we evaluate res in σ'' : $\text{Val}(\text{res}, \sigma'') = 6$

- Thus, $\text{Val}(\text{abs}(3,9), \sigma) = 6$.

Example 3: FOL Specification and Array Modeling

Context: Exercise 3 – function that fills b with the cumulative sums of a .

Application:

1. **Informal implementation:**

```
sum_prefix(a, b) =  
  i := 0;  
  sum := 0;  
  while (i < a.length) do  
    sum := sum + a[i];  
    b[i] := sum;  
    i := i + 1;  
  done
```

2. **Precondition:** $a.\text{length} = b.\text{length} \quad a.\text{length} > 0$
3. **Postcondition:** $\forall j \in [0, a.\text{length}-1], b[j] = \sum_{k=0}^j a[k]$ In FOL, we could write:

$$\forall j (0 \leq j < a.\text{length} \rightarrow b[j] = \sum_{k=0}^j a[k])$$

where `sum` is a metanotation for summation (in pure FOL, we use induction or a defined summation function).

4. **Semantic evaluation** for $\sigma = \{a \leftarrow [0,3,5,6], b \leftarrow [0,0,0,0]\}$:

- Initially $i=0, \text{sum}=0$.
- Iteration 1: $\text{sum}=0+0=0, b[0]=0, i=1$
- Iteration 2: $\text{sum}=0+3=3, b[1]=3, i=2$
- Iteration 3: $\text{sum}=3+5=8, b[2]=8, i=3$
- Iteration 4: $\text{sum}=8+6=14, b[3]=14, i=4$
- Final state: $\sigma' = \{a \leftarrow [0,3,5,6], b \leftarrow [0,3,8,14], i \leftarrow 4, \text{sum} \leftarrow 14\}$

Synthesis

The exercises implement several key methods in software verification: syntax analysis (grammar and context), operational semantics (step-by-step evaluation with memory states), and formal modeling with first-order logic and data structures (arrays). Mastering these methods enables detecting errors, precisely specifying expected behavior, and reasoning about program execution.